

Learn Python for Automation

Lesson 2: Variables, Data Types & Operators

Variables, Data Types & Operators - Study Notes

Key Concepts

1. **Variables** – Named storage locations that hold data values.
2. **Data Types** – Classifications of data (e.g., `int`, `float`, `str`, `list`, `dict`, `bool`).
3. **Operators** – Symbols that perform operations on variables or values (`+`, `-`, `*`, `/`, `==`, `and`, `or`, etc.).
4. **Type Conversion** – Explicitly changing a value from one type to another (`int()`, `float()`, `str()`, etc.).
5. **Mutable vs. Immutable** – Objects that can be changed after creation (`list`, `dict`) vs. those that cannot (`int`, `str`, `tuple`).

Summary

Variables are the backbone of any program; they let you store and manipulate data. In Python, a variable name is simply a reference to an object stored in memory. The language automatically infers the type of a variable based on the value assigned to it, but you can also explicitly cast types when needed.

Data types determine what operations are allowed. Numbers (`int`, `float`) support arithmetic operators, while strings support concatenation and repetition. Collections like lists and dictionaries allow you to group multiple values, each with its own set of operations (e.g., indexing, slicing, membership tests).

Operators are the tools that let you combine or transform these values. Arithmetic operators (`+`, `-`, `*`, `/`, `%`, `**`) work on numbers; comparison operators (`==`, `!=`, `<`, `>`, `<=`, `>=`) return booleans; logical operators (`and`, `or`, `not`) combine boolean values. Understanding how these operators interact with different data types is essential for writing correct and efficient code.

Important Points

- **Variable Naming Rules**

- Must start with a letter or underscore (`_`).
- Can contain letters, digits, and underscores.
- Case sensitive (`count` vs `Count`).
- Avoid using Python keywords (`def`, `class`, `if`, etc.).

- **Data Type Characteristics**

Type	Mutability	Typical Ops	Example
<code>int</code>	Immutable	<code>+</code> , <code>-</code> , <code>*</code> , <code>/</code>	<code>x = 5</code>
<code>float</code>	Immutable	<code>+</code> , <code>-</code> , <code>*</code> , <code>/</code>	<code>y = 3.14</code>
<code>str</code>	Immutable	<code>+</code> , <code>*</code> , <code>in</code> , <code>len()</code>	<code>s = "hello"</code>

`list`	Mutable	`append()`, `pop()`, indexing	`l = [1, 2, 3]`
`dict`	Mutable	`keys()`, `values()`, `get()`	`d = {'a': 1}`
`bool`	Immutable	`and`, `or`, `not`	`b = True`

- **Operator Precedence** (highest != lowest)

1. `\*\*`
2. `+`, `-` (unary)
3. `\*`, `/`, `%`
4. `+`, `-` (binary)
5. Comparisons (`<`, `<=`, `>`, `>=`, `==`, `!=`)
6. `not`
7. `and`
8. `or`

- **Type Conversion**

```
```python
```

```
a = "10"
```

```
b = int(a) # 10 (int)
```

```
c = float(b) # 10.0 (float)
```

```
d = str(c) # "10.0" (str)
```

```
```
```

- **Mutable vs. Immutable**

- Modifying a mutable object changes the original reference.
- Reassigning an immutable variable creates a new object.

Examples

1. Arithmetic with Mixed Types

```
```python
```

```
age = 25 # int
```

```
height = 1.78 # float
```

```
message = "Age: " + str(age) + ", Height: " + str(height)
```

```
print(message) # Age: 25, Height: 1.78
```

```
```
```

Explanation:

- `age` is an integer; `height` is a float.
- To concatenate with a string, each number is cast to `str`.
- The resulting `message` is a single string.

2. List Manipulation & Logical Checks

```
```python
```

```
fruits = ["apple", "banana", "cherry"]
```

```
if "banana" in fruits and len(fruits) > 2:
```

```
 fruits.append("date")
```

```
print(fruits)
```

```

```

**\*Explanation:\***

- ``in`` checks membership (returns ``True``).
- ``len()`` gives the number of items.
- ``and`` ensures both conditions are true before appending.
- Output: ``['apple', 'banana', 'cherry', 'date']``.

```

```

## Quick Review

1. What is the difference between a mutable and an immutable data type?
2. How would you convert the string ``"123"`` to an integer?
3. Which operator has higher precedence: ``*`` or ``+``?
4. Write a line of code that checks if the variable ``x`` is greater than 10 **and** less than 20.
5. Explain why ``a = [1, 2, 3]`` and ``b = a`` will share the same underlying list.

```

```

## Further Reading

- **\*\*Control Flow\*\*** – ``if``, ``elif``, ``else``, and loops (``for``, ``while``).
- **\*\*Functions\*\*** – Defining and calling functions, scope, return values.
- **\*\*Error Handling\*\*** – ``try``, ``except``, ``finally``.
- **\*\*List Comprehensions\*\*** – Concise list creation.
- **\*\*Dictionary Methods\*\*** – ``update()``, ``pop()``, ``get()``, ``keys()``, ``values()``.

These topics build on the fundamentals of variables, data types, and operators, enabling you to write more complex and robust Python programs.

